

Governance Fix Package G3

QA visibility + workspace gating + team filtering + mobile visit-create simplification + defect orphan audit.
Five-item governance sprint. Implementation report. No schema migration. All four validation checks pass.

STATUS — Implemented and validated. API build, admin typecheck, admin build, mobile typecheck all PASS on first attempt. QA actors now bypass team-membership filters on the SiteVisits / Dashboard / Defects / Operations Board read paths and are scoped to their assigned MAINHEADs. Mobile visit creation no longer exposes a team picker. Defect orphan audit script delivered for the connected DB — production probe ready.

1. QA Visibility

Added shared ScopeContext helper and made every read access scope consult it. ADMIN remains unbounded. A QA actor (`isQaActor() = org.type=ASCURE + active QA_VALIDATION` capability) bypasses the team filter and is instead scoped to MAINHEADs the user has explicit access to via `UserMainheadAccess + UserOperationalRegionAccess`.

File	Status	Change
<code>apps/api/src/common/authorization/scope-context.ts</code>	new	Exports <code>ScopeContext</code> and <code>buildScopeContext(prisma, user)</code> . Resolves <code>isQa</code> , <code>isAdmin</code> , and <code>qaMainheadIds</code> from direct + region-derived MAINHEAD access.
<code>apps/api/src/site-visits/site-visits.service.ts</code>	edited	<code>accessScope(user, ctx?)</code> now returns <code>{ mainheadId: { in: ctx.qaMainheadIds } }</code> for QA actors. Public handlers <code>list</code> , <code>getById</code> , <code>getReadById</code> precompute <code>ctx.buildListWhere</code> threads <code>ctx</code> through.
<code>apps/api/src/dashboard/dashboard.service.ts</code>	edited	<code>siteVisitAccessScope</code> and <code>inspectionAccessScope</code> now accept <code>ctx</code> + apply MAINHEAD scope for QA. <code>getDashboard</code> precomputes <code>ctx</code> once and threads it into every <code>accessible*Where</code> call. <code>ensureDefectsForAccessibleItems</code> accepts <code>ctx</code> .
<code>apps/api/src/defects/defects.service.ts</code>	edited	<code>inspectionAccessScope</code> mirrored. <code>list</code> , <code>getOperationsBoard</code> , <code>buildOperationsBoardWhere</code> , <code>ensureDefectsForAccessibleItems</code> , <code>findOrCreateAccessibleDefect</code> , <code>findAccessibleDefectById</code> , <code>findAccessibleDefectByItemResultId</code> all updated.

Access-scope semantics after G3:

```
ADMIN : {} // full tenant visibility
QA actor: { mainheadId: { in: qaMainheadIds } }
Other : { team: { members: { some: { userId, isActive: true } } } }
```

2. Workspace Filtering

No code change required. Verified in `apps/mobile/src/operationalWorkspace.ts:56–82`:

- `getAvailableMobileWorkspaces` renders the Inspection workspace only when the user is ADMIN or has the INSPECTION capability code.
- Maintenance workspace shown only for ADMIN or MAINTENANCE capability holders.
- `getAutoOpenWorkspace` auto-enters the single workspace when exactly one is available — single-workspace users skip the picker entirely.

3. Team dropdown filtered by Organization

User Create/Edit form: team dropdown now filters by the selected Organization. When no organization is picked, the full list is shown.

File	Change
apps/admin-web/src/components/users-client.tsx	Team <select> now wraps the iteration in <code>.filter(team => !values.organizationId !team.organizationId team.organizationId === values.organizationId)</code> . Comment explains the G3 intent. Behaviour preserved when org isn't picked.

Site Visits create form left as-is: it has no Organization selector, and MAINHEAD is already required by Governance G2. The team picker on the mobile checkin flow is removed entirely under Item 4.

4. Mobile Visit Creation — team inferred

CheckInScreen previously showed a multi-team picker ("Change Assigned Team"). G3 removes it; the team is inferred from the user's team membership as returned by GET /users/me/teams. The selected team is shown read-only as Team (Auto).

File	Change
apps/mobile/src/screens/CheckInScreen.tsx	Removed the multi-team picker block (Pressable + SelectCard list). Removed the isTeamPickerOpen state. The auto-selection logic that picked the first available team at loadOptions is retained. The user can no longer switch teams from the mobile UI.

api.getTeams(token) already calls /users/me/teams (apps/mobile/src/api.ts:270–272) so the source list is already scoped to the user's assignments. The UX behaviour matches V10 §Mobile Workspaces: zero-input team binding for field execution.

5. Defect Orphan Audit (assessment-only)

A read-only Prisma script was authored, executed against the connected dev DB, and then deleted (no app code modified). Output for the connected DB:

Tenant	Flagged InspectionItemResult	Defect rows	Orphans
1ef638af-... (demo-tenant — Demo Utility Tenant)	0	0	0

The connected DB has no flagged items to orphan. Production must be probed by the same script — pointed at production DATABASE_URL. The script captures, per tenant: total flagged, total Defect rows, orphan count, and up to 100 orphan detail rows (InspectionItemResult id, inspection id, site visit id + pencawang, asset). It also prints a one-shot idempotent backfill SQL:

```
INSERT INTO "Defect" (...)  
SELECT gen_random_uuid(), iir.id, 'OPEN',  
       COALESCE(iir.severity, 'MEDIUM'), 'DETECTED', NOW(), NOW()  
FROM "InspectionItemResult" iir  
LEFT JOIN "Defect" d ON d."inspectionItemResultId" = iir.id  
WHERE iir."isDefect" = true AND d.id IS NULL  
ON CONFLICT ("inspectionItemResultId") DO NOTHING;
```

Validation

Step	Command	Result
API build	pnpm --filter @ascure/api exec tsc -p tsconfig.build.json	PASS — exit 0
Admin typecheck	pnpm --filter @ascure/admin-web typecheck	PASS — exit 0
Admin build	pnpm --filter @ascure/admin-web build	PASS — all 21 routes generated
Mobile typecheck	(cd apps/mobile && pnpm exec tsc --noEmit)	PASS — exit 0

Pilot risks discovered

- **QA actor with zero accessible MAINHEADs sees zero visits / defects.** The MAINHEAD scope becomes `{ in: [] }`, which Prisma treats as no match. Pre-G3, the same user would see zero via team scope; net behaviour for an un-provisioned QA is unchanged but now explicit. Provision QA users with either direct MAINHEAD or region access before relying on the bypass.
- **Mobile users with multiple teams can no longer choose.** The first team in `/users/me/teams` wins. If product needs explicit primary-team selection, add a `User.primaryTeamId` column in a future migration and have the API return it first.
- **Two near-duplicate `ensureDefectsForAccessibleItems` copies remain** (`dashboard.service.ts` + `defects.service.ts`). G3 brings them up to feature parity; consolidation is a follow-up.
- **QA visibility refactor is scoped to read paths only.** Mutation paths (`verify`, `close`, `assign`) still use `findAccessibleDefectById` / `findAccessibleDefectByItemId` which now also consult `ctx` — so QA actors can act on defects within their MAINHEAD scope. `findAccessibleSiteVisit` still uses legacy team scope for write-side mutations (`uploadImage`, `complete`, `cancel`, `link asset`). QA does not currently mutate site visits directly; flag for follow-up if that changes.

Rollout order

- **API first.** The scope-context + access-scope changes are source-of-truth.
- **Admin web second.** User-form team filter only takes effect once shipped; API is forward-compatible either way.
- **Mobile third.** Build + deploy. The `CheckInScreen` change is purely UI; the `POST /site-visits` payload shape is unchanged.
- **Production probe before deploy:** run the §5 audit script against production. If orphans > 0, run the printed backfill SQL inside a single transaction. Re-run the audit; orphan count should drop to zero.